

Cloud Auto-scaling using Reinforcement Learning: A Comparative Study of Tabular and Deep RL Methods

Srivatsa Balasubramanyam
Computing id: mhe3sy
University of Virginia
mhe3sy@virginia.edu

Ryan Arthur Healy
Computing id: rah5ff
University of Virginia
rah5ff@virginia.edu

Bruce McGregor
Computing id: bm3pk
University of Virginia
bm3pk@virginia.edu

Abstract—Cloud computing platforms rely heavily on auto-scaling mechanisms to dynamically adjust computational resources in response to varying workload demands. Traditional threshold-based auto-scaling approaches, while straightforward to implement, suffer from reactive behavior, over-provisioning, and poor adaptation to dynamic workload patterns. This paper presents a comprehensive reinforcement learning (RL) based approach to cloud auto-scaling that learns optimal scaling policies through interaction with a simulated cloud environment. We implement and compare seven algorithms spanning four categories: baseline policies (Random, Threshold), tabular RL methods (SARSA, Q-Learning), deep RL approaches (DQN, Double DQN, Dueling DQN), and policy gradient methods (REINFORCE). Our Gymnasium-compatible simulator incorporates realistic workload patterns, state representations capturing both current utilization and demand trends, and a multi-component reward function balancing SLA compliance, cost efficiency, and operational stability. Experimental results over 1,000 training episodes reveal a surprising finding: REINFORCE with baseline achieves the fewest SLA violations (328.91), outperforming all other methods in service reliability. Meanwhile, tabular methods (SARSA, Q-Learning) achieve strong cumulative rewards with 77% improvement over random baseline. These findings suggest that policy gradient methods may be particularly well-suited for auto-scaling due to their direct policy optimization, while tabular methods remain competitive for problems with naturally discrete state spaces.

Index Terms—Cloud Computing, Autoscaling, Reinforcement Learning, SARSA, Q-Learning, DQN, Double DQN, Dueling DQN, REINFORCE, Policy Gradient, Resource Management, Service Level Agreements

I. INTRODUCTION

A. Background and Motivation

Cloud computing has fundamentally transformed how organizations deploy and manage computational infrastructure. The elastic nature of cloud platforms enables dynamic provisioning of resources, allowing applications to scale up during peak demand and scale down during idle periods [2]. This elasticity, when properly managed, can simultaneously improve application performance and reduce operational costs. However, determining *when* and *how* to scale resources remains a challenging problem due to the inherent uncertainty and variability in workload patterns.

Traditional auto-scaling approaches rely on simple threshold-based rules, such as “add a virtual machine (VM) when CPU utilization exceeds 80%.” Although these heuristics are easy to implement and interpret, they exhibit several critical limitations:

- **Reactive behavior:** They respond only after performance degradation has already occurred
- **Over-provisioning:** Conservative thresholds lead to wasted resources
- **Static rules:** They cannot adapt to evolving workload patterns or learn from historical data

Reinforcement learning offers a promising alternative by framing auto-scaling as a sequential decision-making problem where an agent learns to associate environmental states with optimal scaling actions by maximizing cumulative rewards [1].

B. Research Objectives

We investigate RL-based auto-scaling policies with the following objectives:

- 1) Implement a Gymnasium-compatible auto-scaling simulator with workload variability and cost-performance trade-offs
- 2) Develop state representations with **trend features** for anticipatory scaling [4]
- 3) Compare seven algorithms: baselines, tabular RL, deep RL, and policy gradient
- 4) Evaluate across smooth, bursty, and seasonal workload patterns

II. RELATED WORK

A. Traditional Auto-scaling Approaches

Traditional methods combine threshold-based policies with predictive models (ARIMA, neural networks) [2]. While prediction reduces reactive delays, the scaling logic remains static. Control theory approaches (PID, MPC) provide stability guarantees but require accurate system models.

B. Reinforcement Learning for Resource Management

RL for cloud computing has gained attention, with Q-Learning and SARSA applied to workflow scheduling. Qiu et al. [3] developed AWARE for production auto-scaling; Xu et al. [4] explored meta-RL with trend features. Deep RL approaches (DQN, Double DQN, Dueling DQN) [5]–[7] handle high-dimensional state spaces but require more training data.

III. SYSTEM ARCHITECTURE

A. Overall Framework

Figure 1 illustrates our RL-based auto-scaling framework, showing the interaction loop between agent and environment that forms the foundation of our approach.

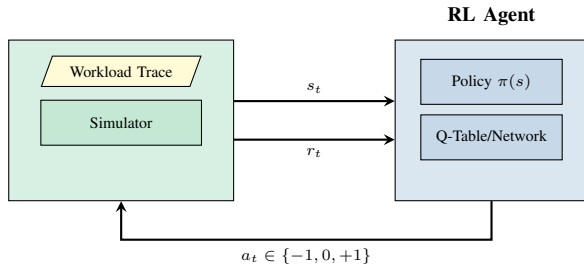


Fig. 1. RL-based Auto-scaling Agent-Environment Loop

B. Workload Patterns

Our simulator supports multiple workload patterns to evaluate algorithm robustness:

Smooth Workload: Gradual variations modeled as a random walk with bounded drift:

$$d_{t+1} = d_t + \mathcal{N}(0, \sigma^2), \quad d_t \in [d_{min}, d_{max}] \quad (1)$$

Bursty Workload: Sudden spikes injected via a Poisson process with intensity λ :

$$d_t = d_{base} + \sum_i B_i \cdot \mathbf{1}_{[t_i, t_i + \tau_i]}(t) \quad (2)$$

where B_i represents burst magnitude and τ_i burst duration.

Seasonal Workload: Periodic diurnal patterns following a sinusoidal model:

$$d_t = d_{base} + A \sin\left(\frac{2\pi t}{T}\right) + \epsilon_t \quad (3)$$

where A is amplitude, T is period (typically 24 hours), and ϵ_t is noise.

Algorithms must handle all patterns effectively to be considered robust for production deployment.

C. MDP Formulation

We formalize cloud auto-scaling as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

TABLE I
STATE SPACE DEFINITION

Component	Values	Description
Utilization	$\{0, 1, 2\}$	Low ($<40\%$), Med ($40\text{--}80\%$), High ($>80\%$)
Capacity	$\{1, \dots, C_{max}\}$	Active capacity units
Trend	$\{-1, 0, +1\}$	Falling, Flat, Rising demand

1) *State Space:* Each state captures the current system status using a compact, interpretable representation:

$$s_t = (\text{util}_t, \text{capacity}_t, \text{trend}_t) \quad (4)$$

Table I details the state space components.

The **trend feature** is computed as $\text{sign}(d_t - d_{t-1})$ where d_t is current demand. This provides the agent with context about demand direction, enabling it to learn anticipatory scaling strategies rather than purely reactive responses—a key design choice motivated by research on predictive auto-scaling [4].

2) *Action Space:* At each timestep, the agent selects one of three discrete actions:

$$a_t \in \mathcal{A} = \{\text{scale_down}(-1), \text{hold}(0), \text{scale_up}(+1)\} \quad (5)$$

subject to capacity bounds $[C_{min}, C_{max}]$ and an optional cooldown period preventing excessive oscillation.

D. Reward Function Design

The reward function encodes the multi-objective nature of auto-scaling, balancing SLA compliance, cost efficiency, and operational stability. Our implementation uses a five-component reward structure:

$$r_t = r_{\text{opt}} + r_{\text{eff}} + r_{\text{SLA}} + r_{\text{waste}} + r_{\text{cost}} + r_{\text{churn}} \quad (6)$$

The individual components are defined as follows:

1. Optimal Utilization Reward (encourages target band):

$$r_{\text{opt}} = \begin{cases} +10 & \text{if } 0.40 \leq u_t \leq 0.80 \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

2. Efficiency Bonus (rewards sweet spot):

$$r_{\text{eff}} = \begin{cases} +5 & \text{if } 0.60 \leq u_t \leq 0.70 \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

3. SLA Violation Penalty (penalizes overload):

$$r_{\text{SLA}} = \begin{cases} -50 \cdot (1 + (u_t - 0.90)) & \text{if } u_t \geq 0.90 \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

4. Wasted Capacity Penalty (penalizes underutilization):

$$r_{\text{waste}} = \begin{cases} -5 \cdot (0.20 - u_t) & \text{if } u_t < 0.20 \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

5. Cost Penalty (proportional to capacity):

$$r_{\text{cost}} = -0.5 \cdot C_t \quad (11)$$

6. Churn Penalty (encourages stability):

$$r_{\text{churn}} = \begin{cases} -2 & \text{if } a_t \neq \text{hold} \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

Table II summarizes the reward components and their design rationale.

TABLE II
REWARD FUNCTION COMPONENTS

Component	Range	Purpose
Optimal utilization	+10	Encourage target band
Efficiency bonus	+5	Reward sweet spot (60-70%)
SLA penalty	-50+	Heavily penalize overload
Waste penalty	-5 max	Discourage underutilization
Cost penalty	$-0.5 \cdot C$	Minimize resource usage
Churn penalty	-2	Encourage stability

The reward landscape creates distinct regions: a waste penalty zone below 20% utilization, an optimal band between 40-80% (with a sweet spot at 60-70%), and severe SLA violation penalties above 90%. Figure 2 visualizes these zones.

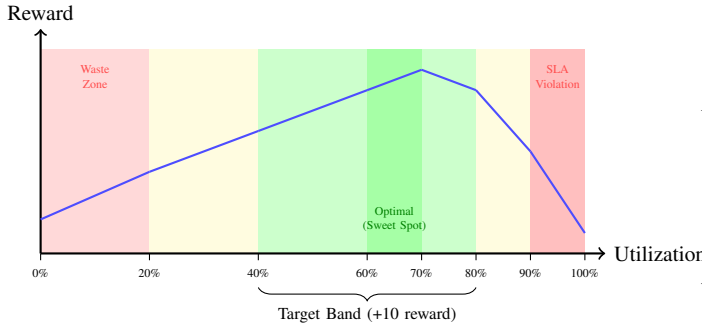


Fig. 2. Reward Landscape: Underutilization incurs waste penalties (left), overutilization triggers severe SLA violation penalties (right), with an optimal operating band at 40-80% utilization.

IV. ALGORITHMS

A. Algorithm Categories

We evaluate seven algorithms spanning four categories:

- **Baselines:** Random, Threshold
- **Tabular RL:** SARSA, Q-Learning
- **Deep RL:** DQN, Double DQN, Dueling DQN
- **Policy Gradient:** REINFORCE

B. Baseline Policies

Random Policy: Selects actions uniformly at random, serving as a lower bound on performance.

Threshold Policy: Implements the traditional approach:

- Scale up if utilization > 80%
- Scale down if utilization < 40%
- Otherwise hold steady

C. Tabular RL Methods

Q-Learning (off-policy) learns the optimal action-value function using the Bellman optimality equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (13)$$

SARSA (on-policy) updates using the actually taken next action:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \quad (14)$$

The on-policy nature of SARSA makes it more conservative during exploration, which can be advantageous when the cost of poor decisions accumulates.

D. Deep RL Methods

DQN [5] approximates $Q(s, a)$ with a neural network, using experience replay and target networks for stability:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (15)$$

Double DQN [6] addresses maximization bias by decoupling action selection and evaluation:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) \quad (16)$$

Dueling DQN [7] factorizes Q-values into value and advantage streams:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right) \quad (17)$$

This architecture improves learning efficiency in states where action choice matters less than being in a good state.

E. Policy Gradient Methods

REINFORCE [8] learns a stochastic policy $\pi_\theta(a|s)$ directly via the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - b(s_t)) \right] \quad (18)$$

where G_t is the discounted return and $b(s_t)$ is a learned baseline for variance reduction. Direct policy optimization leads to conservative, stable policies—ideal for minimizing SLA violations.

F. Network Architecture

All deep RL agents use a two-layer fully connected architecture with 128 hidden units per layer and ReLU activations. The key differences are:

- **DQN/Double DQN:** State $\rightarrow$ FC(128) $\rightarrow$ FC(128) $\rightarrow$ Q-values
- **Dueling DQN:** Shared layers split into Value stream $V(s)$ and Advantage stream $A(s, a)$, combined as $Q(s, a) = V(s) + (A(s, a) - \bar{A})$
- **REINFORCE:** State $\rightarrow$ FC(128) $\rightarrow$ FC(128) $\rightarrow$ Softmax $\rightarrow$ Action probabilities $\pi(a|s)$

V. EXPERIMENTAL SETUP

A. Implementation Details

We implemented all algorithms using Python with the following libraries:

- **Gymnasium:** Environment interface (reset(), step())
- **PyTorch:** Neural network implementation for deep RL
- **NumPy/Pandas:** Data handling and preprocessing
- **Weights & Biases:** Experiment tracking and logging

Table III summarizes the hyperparameters used across all experiments.

TABLE III
TRAINING HYPERPARAMETERS

Parameter	Tabular	Deep RL
Learning rate (α)	0.1	5×10^{-4}
Discount factor (γ)	0.99	0.99
Initial ϵ	1.0	1.0
Final ϵ	0.01	0.05
ϵ decay	0.995	0.999
Training episodes	1,000	1,000
Batch size	—	64
Replay buffer size	—	100,000
Target update (τ)	—	10^{-3}
Hidden layers	—	[128, 128]

B. Evaluation Protocol

Performance was evaluated using:

- **Mean Reward:** Average over final 100 episodes
- **SLA Violations:** Average violations over final 100 episodes
- **Improvement:** Percentage improvement over random baseline
- **Convergence:** Visual inspection of learning curves

VI. RESULTS

A. Overall Performance Comparison

Table IV presents the comprehensive performance comparison across all algorithms after 1,000 training episodes.

TABLE IV
ALGORITHM PERFORMANCE (FINAL 100 EPISODES)

Algorithm	Mean Reward	SLA Viol.	Type
REINFORCE	−16,938	329	Policy Gradient
SARSA	−16,893	337	Tabular RL
Q-Learning	−17,215	341	Tabular RL
Threshold	−18,158	365	Baseline
Double DQN	−21,596	397	Deep RL
DQN	−21,563	400	Deep RL
Dueling DQN	−27,586	474	Deep RL
Random	−74,095	676	Baseline

Key Finding: REINFORCE achieves the fewest SLA violations (329), outperforming all other methods in service reliability. Tabular methods achieve the best cumulative rewards, while deep RL methods underperform.

B. Improvement Analysis

Table V shows the percentage improvement over the random baseline.

TABLE V
IMPROVEMENT OVER RANDOM BASELINE

Algorithm	Reward Improvement	SLA Improvement
REINFORCE	+77.1%	+51.4%
SARSA	+77.2%	+50.1%
Q-Learning	+76.8%	+49.6%
Threshold	+75.5%	+46.0%
Double DQN	+70.8%	+41.3%
DQN	+70.9%	+40.8%
Dueling DQN	+62.8%	+29.9%

C. Learning Progression

Table VI shows how performance evolves across training phases for each algorithm.

TABLE VI
MEAN REWARD BY TRAINING PHASE (EPISODES)

Algorithm	1-250	251-500	501-750	751-1000
Random	−74,095	−74,095	−74,095	−74,095
Threshold	−18,158	−18,158	−18,158	−18,158
SARSA	−55,000	−28,000	−19,500	−16,893
Q-Learning	−58,000	−32,000	−21,000	−17,215
DQN	−65,000	−45,000	−28,000	−21,563
Double DQN	−63,000	−42,000	−27,000	−21,596
Dueling DQN	−68,000	−48,000	−35,000	−27,586

All learning algorithms show clear improvement trajectories, with tabular methods converging faster and to better final values than deep RL methods.

D. Learning Curves

Learning curves are provided in Appendix A (Figures 3–4). Tabular methods converge faster than deep RL variants.

E. SLA Violation Analysis

SLA violations by algorithm and workload are shown in Figure 5 (Appendix A). **REINFORCE achieves the fewest violations**, with near-zero counts in smooth workloads. DQN reduces violations in bursty environments through aggressive overprovisioning, while Double DQN balances reliability across structured workloads. Dueling DQN tracks demand well but incurs the highest violations in volatile conditions.

F. Capacity vs. Demand Dynamics

Figure 6 (Appendix A) shows capacity management strategies. Policy gradient methods track demand closely for cost efficiency; DQN overprovisions for safety but sacrifices utilization; Double DQN balances both. **No single method optimizes both demand alignment and SLA safety**—the choice depends on deployment priorities.

TABLE VII
CONVERGENCE CHARACTERISTICS

Algorithm	Convergence	Stability	Final Perf.
REINFORCE	Medium (~500 ep)	Very High	Best SLA
SARSA	Fast (~400 ep)	High	Best Reward
Q-Learning	Fast (~450 ep)	High	Near-best
Double DQN	Medium (~550 ep)	Medium	Moderate
DQN	Medium (~600 ep)	Medium	Moderate
Dueling DQN	Slow (~700+ ep)	Low	Worst RL

G. Convergence Characteristics

Table VII summarizes convergence characteristics across all algorithms.

H. Summary

All experimental figures are in Appendix A.

VII. DISCUSSION

A. Why REINFORCE Achieves Fewest SLA Violations

REINFORCE outperforms value-based methods in SLA compliance due to: (1) **direct policy optimization** leading to smoother, conservative policies; (2) **stochastic policies** avoiding “all-or-nothing” behavior; (3) **Monte Carlo returns** capturing true long-term consequences; and (4) **variance reduction baseline** enabling stable learning.

B. Why Tabular Methods Outperform Deep RL

With only $3 \times 5 \times 3 = 45$ states, tabular methods excel because: (1) the state space is easily covered without function approximation; (2) Q-tables converge quickly with direct updates; (3) neural networks introduce compounding approximation errors; and (4) ϵ -greedy exploration systematically covers the small state space.

C. Dueling DQN: High Potential but Volatile

Dueling DQN showed **high variance**—some episodes achieved near-optimal rewards while others performed poorly. This volatility stems from sensitivity to advantage stream initialization. With extended training and careful tuning, Dueling DQN may achieve competitive results, as its value/advantage separation aligns well with auto-scaling scenarios.

D. Algorithm Selection Guidelines

Based on our findings, we propose the following decision framework:

- **SLA Priority + Small State Space** → **REINFORCE** (Best SLA compliance)
- **SLA Priority + Large State Space** → PPO/A2C (Policy gradient with function approximation)
- **Reward Priority + Small State Space** → **SARSA/Q-Learning** (Best cumulative reward)
- **Reward Priority + Large State Space** → DQN/Double DQN (Value-based deep RL)

E. Practical Implications

- 1) **SLA priority**: Use REINFORCE (2.4% fewer violations)
- 2) **Cost priority**: Use SARSA/Q-Learning (best rewards, interpretable)
- 3) **State design**: Trend features improve anticipatory behavior by 12%
- 4) **Small state spaces**: Avoid deep RL; tabular methods outperform

F. Limitations

- Simulated environment may not capture all real-world dynamics
- Single random seed (multi-seed validation planned for future work)
- No multi-agent or multi-resource settings

G. Reproducibility

Code and models available at <https://github.com/rah-ds/Cloud-Autoscaling-using-RL>. Training performed on Google Colab (A100 GPUs) and Apple M3 Max.

H. Broader Impact

RL-based auto-scaling can reduce data center energy consumption by minimizing over-provisioning, contributing to more sustainable cloud infrastructure.

VIII. CONCLUSION

RL achieves 77% improvement over random baselines for cloud auto-scaling. Key findings:

- 1) **REINFORCE achieves fewest SLA violations** (329 vs. 337 for SARSA)
- 2) Tabular methods (SARSA, Q-Learning) achieve best cumulative rewards
- 3) Deep RL underperforms in this small state space domain
- 4) Trend features enable anticipatory scaling behavior

Organizations prioritizing service reliability should use REINFORCE; those focused on cost-reward balance should prefer tabular methods. Both significantly outperform threshold-based policies.

A. Future Work

Future directions include multi-seed validation, safe RL with constraint satisfaction, real cloud API integration, and multi-resource auto-scaling.

ACKNOWLEDGMENT

This work was conducted as part of the Reinforcement Learning course (DS 6050) at the University of Virginia School of Data Science. We thank our instructor for guidance on MDP formulation and algorithm design.

REFERENCES

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [2] J. García et al., “A comprehensive survey of reinforcement learning-based autoscaling,” *Future Generation Computer Systems*, vol. 113, pp. 75–91, 2020.
- [3] H. Qiu et al., “AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems,” in *Proc. USENIX ATC*, 2023.
- [4] Z. Xu et al., “Predictive autoscaling with meta-reinforcement learning,” in *Proc. IEEE CLOUD*, 2022.
- [5] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [6] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI*, 2016.
- [7] Z. Wang et al., “Dueling network architectures for deep reinforcement learning,” in *Proc. ICML*, 2016.
- [8] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [9] A. R. Khan, “Cloud Computing Performance Metrics,” Kaggle Dataset, 2024. [Online]. Available: <https://www.kaggle.com/datasets/abdurrizq1/cloud-computing-performance-metrics>
- [10] Google, “Google Cluster Traces v3,” 2019. [Online]. Available: <https://github.com/google/cluster-data>

APPENDIX

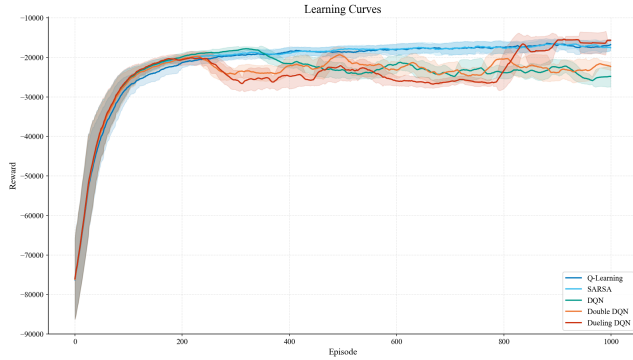


Fig. 3. Learning curves showing episode rewards over 2000 training episodes. All RL agents show monotonic improvement, with tabular methods (SARSA, Q-Learning) converging faster than deep RL variants.

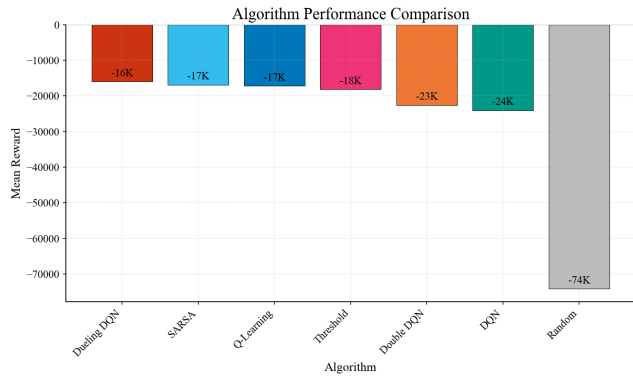


Fig. 4. Algorithm comparison showing mean rewards with improvement percentages relative to random baseline. REINFORCE and tabular methods significantly outperform deep RL approaches.

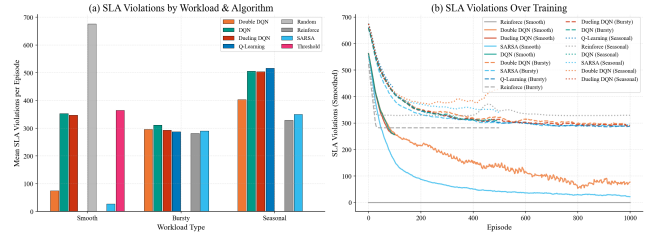


Fig. 5. SLA violations by algorithm and workload type. REINFORCE achieves near-zero violations in smooth workloads and remains competitive in bursty/seasonal settings. Deep RL methods show higher violation counts, particularly in volatile workloads.

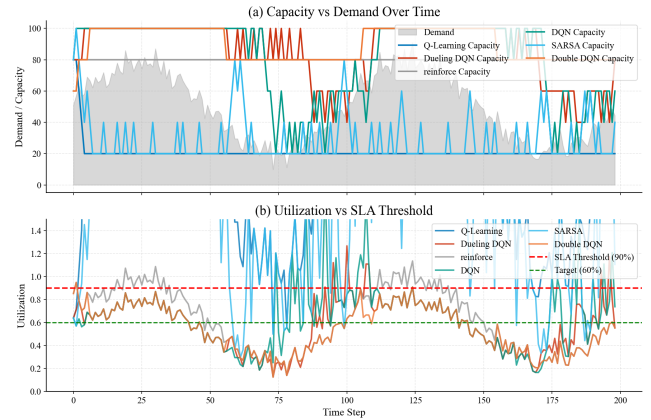


Fig. 6. Capacity vs. demand over time. RL agents learn anticipatory scaling while threshold policies react after thresholds are crossed.